

Homework.2 - Object Detection

- Student name: 林浩君
- Student ID: 312552014

Script

- Training: `python -m yolox.tools.train -n yolox-s -d 1 -b 8 --fp16 -o -c yolox_s.pth --cache`
- Validating: `python -m yolox.tools.eval -n yolox-s -c YOLOX_outputs/yolox_s/best_ckpt.pth -d 1 -b 8`
- Testing: `python -m yolox.tools.demo image -n yolox-s -c YOLOX_outputs/yolox_s/best_ckpt.pth --path ./datasets/COCO/test --save_result`

Questions

- Experiment Setup
 - Data Preprocessing
 1. download dataset from google drive and renamed as `COCO`
 2. rename `train` to `train2017`, `val` to `val2017`
 3. write a convert code to convert yolov8 txt format into coco json format

```
# convert.py
''' Your folder structure should look like this
COCO
├── annotations
│   ├── instances_train2017.json
│   └── instances_val2017.json
├── test
├── train2017
└── val2017
'''

import json
from pathlib import Path

def generate_coco_format(mode="train"):
    image_width = 1920
    image_height = 1080

    # Start building the COCO dataset structure.
    coco_format = {
        "images": [],
        "annotations": [],
    }
```

```

coco_format["categories"] = [{
    "id": 0,
    "name": "car"
}]

# Assuming you have the same structure of the dataset.
dataset_path = Path(".")
images_path = dataset_path / f"{mode}2017"
annotations_path = dataset_path / f"{mode}_labels"

# Add image and annotation information
for image_path in images_path.glob("*.jpg"):
    image_id = int(image_path.stem) # Assumes filename is the ID

    # Add image information
    coco_format["images"].append({
        "id": image_id,
        "file_name": image_path.name,
        "width": image_width, # You need to get the image width
        "height": image_height # You need to get the image height
    })

# Read the YOLO format annotations
annotation_path = annotations_path / f"{image_path.stem}.txt"
with annotation_path.open() as file:
    for i, line in enumerate(file, 1):
        class_id, x_center, y_center, width, height = map(float, line.split())

        # Convert to absolute coordinates and then to COCO format
        x_center_abs = x_center * image_width
        y_center_abs = y_center * image_height
        width_abs = width * image_width
        height_abs = height * image_height

        x_min = x_center_abs - (width_abs / 2)
        y_min = y_center_abs - (height_abs / 2)

        annotation_item = {
            "id": len(coco_format["annotations"]) + 1,
            "image_id": int(image_id),
            "category_id": int(class_id),
            "bbox": [int(x_min), int(y_min), int(width_abs), int(height_abs)],
            "area": int(width_abs * height_abs),
            "iscrowd": 0,
            "segmentation": [] # If you have segmentation info, include it here
        }
        coco_format["annotations"].append(annotation_item)

# Save the COCO data to a file
with open(f"annotations/instances_{mode}2017.json", 'w') as outfile:
    json.dump(coco_format, outfile, indent=4)

if __name__ == "__main__":
    generate_coco_format(mode="train")
    generate_coco_format(mode="val")

```

4. Then go to your YOLOX folder, soft link the processed dataset

- `ln -s /path/to/COCO ./datasets/COCO`

- Hyperparameters - same as default

keys	values
seed	None
output_dir	'./YOLOX_outputs'
print_interval	10
eval_interval	10
dataset	<yolox.data.datasets.coco.COCODataset object at 0x7f8e523658d0>
num_classes	80
depth	0.33
width	0.5
act	'silu'
data_num_workers	4
input_size	(640, 640)
multiscale_range	5
data_dir	None
train_ann	'instances_train2017.json'
val_ann	'instances_val2017.json'
test_ann	'instances_test2017.json'
mosaic_prob	1.0
mixup_prob	1.0
hsv_prob	1.0
flip_prob	0.5
degrees	10.0
translate	0.1
mosaic_scale	(0.1, 2)
enable_mixup	True
mixup_scale	(0.5, 1.5)
shear	2.0

warmup_epochs	5
max_epoch	300
warmup_lr	0
min_lr_ratio	0.05
basic_lr_per_img	0.00015625
scheduler	'yoloxwarmcos'
no_aug_epochs	15
ema	True
weight_decay	0.0005
momentum	0.9
save_history_ckpt	True
exp_name	'yolox_s'
test_size	(640, 640)
test_conf	0.01
nmsthre	0.65

- Explain which layer you add SE modules to and compare the corresponding results
 - I add on SE modules on `cls_convs` and `reg_convs`

```
(head): YOLOXHead(
  (cls_convs): ModuleList(
    (0-2): 3 x Sequential(
      (0): BaseConv(
        (conv): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
        (bn): BatchNorm2d(128, eps=0.001, momentum=0.03, affine=True, track_running_stats=True)
        (act): SiLU(inplace=True)
      )
      (1): SELayer(
        (avg_pool): AdaptiveAvgPool2d(output_size=1)
        (fc): Sequential(
          (0): Linear(in_features=128, out_features=8, bias=True)
          (1): ReLU(inplace=True)
          (2): Linear(in_features=8, out_features=128, bias=True)
          (3): Sigmoid()
        )
      )
    )
    (2): BaseConv(
      (conv): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
```

```

        (bn): BatchNorm2d(128, eps=0.001, momentum=0.03, affine=True, track_running_stats=True)
        (act): SiLU(inplace=True)
    )
)
)
(reg_convs): ModuleList(
  (0-2): 3 x Sequential(
    (0): BaseConv(
      (conv): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn): BatchNorm2d(128, eps=0.001, momentum=0.03, affine=True, track_running_stats=True)
      (act): SiLU(inplace=True)
    )
    (1): SELayer(
      (avg_pool): AdaptiveAvgPool2d(output_size=1)
      (fc): Sequential(
        (0): Linear(in_features=128, out_features=8, bias=True)
        (1): ReLU(inplace=True)
        (2): Linear(in_features=8, out_features=128, bias=True)
        (3): Sigmoid()
      )
    )
    (2): BaseConv(
      (conv): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn): BatchNorm2d(128, eps=0.001, momentum=0.03, affine=True, track_running_stats=True)
      (act): SiLU(inplace=True)
    )
  )
)
# Remaining Modules ...
)

```

- The result seems no effect before and after adding SE module, complete log shown below.
- Screenshot your validation results on your two models (with / without SE module)
 - with SE module

```

$ python -m yolox.tools.eval -n yolox-s -c YOLOX_outputs/yolox_s/best_ckpt.pth -d 1 -b 8

2023-11-08 19:07:27 | INFO      | __main__:204 -
Average forward time: 12.33 ms, Average NMS time: 0.55 ms, Average inference time: 12.89 ms
Average Precision (AP) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.925
Average Precision (AP) @[ IoU=0.50      | area=   all | maxDets=100 ] = 0.989
Average Precision (AP) @[ IoU=0.75      | area=   all | maxDets=100 ] = 0.989
Average Precision (AP) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.914
Average Precision (AP) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.907
Average Precision (AP) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.945
Average Recall    (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=  1 ] = 0.150
Average Recall    (AR) @[ IoU=0.50:0.95 | area=   all | maxDets= 10 ] = 0.938
Average Recall    (AR) @[ IoU=0.50:0.95 | area=   all | maxDets=100 ] = 0.940
Average Recall    (AR) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.916
Average Recall    (AR) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.923
Average Recall    (AR) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.956
per class AP:
| class | AP      |
|:-----|:-----|
| car   | 92.488 |

```

```

per class AR:
| class | AR |
|:-----|:-----|
| car | 93.978 |

```

- without SE module

```

$ python -m yolox.tools.eval -n yolox-s -c YOLOX_outputs/yolox_s/best_ckpt.pth -d 1 -b 8

2023-11-08 19:13:14 | INFO | __main__:196 -
Average forward time: 11.74 ms, Average NMS time: 0.54 ms, Average inference time: 12.28 ms
Average Precision (AP) @[ IoU=0.50:0.95 | area= all | maxDets=100 ] = 0.925
Average Precision (AP) @[ IoU=0.50 | area= all | maxDets=100 ] = 0.989
Average Precision (AP) @[ IoU=0.75 | area= all | maxDets=100 ] = 0.989
Average Precision (AP) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.902
Average Precision (AP) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.904
Average Precision (AP) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.946
Average Recall (AR) @[ IoU=0.50:0.95 | area= all | maxDets= 1 ] = 0.151
Average Recall (AR) @[ IoU=0.50:0.95 | area= all | maxDets= 10 ] = 0.937
Average Recall (AR) @[ IoU=0.50:0.95 | area= all | maxDets=100 ] = 0.940
Average Recall (AR) @[ IoU=0.50:0.95 | area= small | maxDets=100 ] = 0.905
Average Recall (AR) @[ IoU=0.50:0.95 | area=medium | maxDets=100 ] = 0.922
Average Recall (AR) @[ IoU=0.50:0.95 | area= large | maxDets=100 ] = 0.957
per class AP:
| class | AP |
|:-----|:-----|
| car | 92.495 |
per class AR:
| class | AR |
|:-----|:-----|
| car | 93.951 |

```

Reference

- <https://github.com/Megvii-BaseDetection/YOLOX>